PROYECTO DE LABORATORIO NO. 1:
Sistema de Parqueo Inteligente
(gestión básica de tickets y cobros)

José Fernando Sandoval Ceijas - 1206726
Diego Enrique Bustamante Chigua - 1049426

Facultad de ingeniería, Universidad Rafael Landívar
Pensamiento Computacional
Ing. Cindy García Pérez
11 de marzo de 2026

1. ACCIONES DEL PROGRAMA

El programa tendrá primero un registro inicial para ver la información general del operador y un menú interactivo para el usuario que permitirá realizar distintas acciones dentro del sistema de parqueo, ejemplos a detalle:

1. **Registro inicial:** Se solicitan los datos del operador, el código de turno y la capacidad que tiene el parqueo.

2. **Crear ticket de entrada:** Se registra el ingreso de un vehículo al parqueo, guardando los datos del vehículo (placa y si es moto, auto o pickup/camioneta), el nombre del cliente, el minuto de entrada al parqueo y marca como activo el ticket.

3. **Registrar salida y calcular cobro:** Si solo existe un único ticket activo, se calcula el tiempo que el vehículo permaneció en el parqueo para poder determinar el monto a pagar según el tipo de vehículo y las reglas que se establecerán. Esta información se irá guardando para la realización del registro inicial.

4. **Ver estado del parqueo:** Se mostrará información general del sistema, así como la capacidad del parqueo, espacios que se encuentran ocupados, tiempo que estuvo en el parqueo (simulación), y posteriormente para el registro inicial, se sumará el monto que ayudará a ir sumando el dinero recaudado y un ticket más creado.

5. **Simular paso del tiempo:** Para pagar, permitir aumentar el tiempo del sistema en minutos para simular el tiempo que los vehículos permanecen estacionados, considerando que si se está 6 horas debe salir una advertencia de multa, y si supera las 12 se muestra advertencia de recargo por permanencia extrema (este recargo equivale a un 20% más sobre el monto total que se pagará) Y también considerar el descuento si el cliente es VIP (10% de descuento)

6. **Salir del sistema:** Finalizar el programa mostrando un resumen final del turno del operador.

2. DATOS DE ENTRADA

El sistema solicitará al usuario distintos datos necesarios para el funcionamiento del programa, tanto si es cliente, como si es operador, se puede dividir precisamente en dos secciones, donde el usuario coloque si es cliente u operador, para así proceder con la solicitud de los datos (pago de parqueo si es cliente, y resumen del turno si es operador). Considerando lo anterior, los datos de entrada de forma general serían los siguientes:

Se solicita el nombre del operador, el cual será almacenado como un dato de tipo *string*, ya que corresponde a texto que identifica a la persona encargada del sistema durante el turno.

También se pedirá el código de turno, que igualmente será de tipo *string*. Este código debe contener exactamente cuatro caracteres, ya que sirve para identificar el turno de trabajo dentro del sistema.

Otro dato que se solicitará es la capacidad del parqueo, que será de tipo *int*, ya que representa un número entero correspondiente a la cantidad total de espacios disponibles en el parqueo.

Al momento de registrar el ingreso de un vehículo, el sistema pedirá la placa del vehículo, la cual será de tipo *string* y deberá contener entre seis y ocho caracteres sin espacios, con el fin de identificar correctamente el vehículo que ingresó.

Además, se solicitará el tipo de vehículo, que será de tipo *int*, donde el usuario deberá seleccionar una opción numérica según el vehículo:
1 para moto, 2 para automóvil sedan y 3 para pickup o camioneta.

También se solicitará el nombre del cliente, que será almacenado como un dato de tipo *string*, ya que corresponde al nombre de la persona propietaria del vehículo.

El sistema pedirá también la cantidad de minutos que se estuvo estacionado, pero para simular precisamente el paso del tiempo dentro del parqueo, que será de tipo *int*. Este valor permitirá aumentar el tiempo del sistema y simular el tiempo que los vehículos permanecen estacionados.

Finalmente, el sistema podrá solicitar si el cliente es VIP, lo cual puede almacenarse como un dato de tipo *bool* o mediante una respuesta de tipo *string*. Esta información permitirá determinar si se aplicará un descuento de VIP (10%) en el cobro final.

3. VARIABLES

El programa utilizará variables simples para almacenar la información del sistema, son bastantes variables, así que se deben considerar nombres de variables no tan largos, pero a la vez que sean exactos y fáciles de identificar cual fue utilizada para qué situación dentro del sistema, para evitar ya sea duplicar variables o confundirse al momento de colocarlas en cada situación, es por ello que no se utilizarán tantas abreviaciones, serán nombres completos.

Variables del sistema de operador.

- nombreOperador (string)
- codigoTurno (string)
- capacidadParqueo (int)
- ticketsCreados (int)
- ticketsCerrados (int)
- dineroRecaudado (double) ● tiempoSimulado (int)

Variables de registro de datos del cliente para generar ticket

- ticketActivo (bool)
- placaVehiculo (string)
- tipoVehiculo (int)
- nombreCliente (string)
- minutoEntrada (int) ● VIP (bool)

Variables utilizadas para cálculos

- minutosEstacionado (int)
- tarifaHora (double)
- montoBase (double)

- multa (double)
- descuento (double)
- recargoExtremo (double)
- montoFinal (double)

4. CONDICIONES Y CÁLCULOS

Condiciones:

1. El código de turno debe tener exactamente 4 caracteres.

2. Si el código ingresado tiene menos o más de 4 caracteres, el sistema debe mostrar un error y volver a solicitar el dato.

3. La capacidad del parqueo debe ser un número entero mayor o igual a 10.

4. Si el valor ingresado es menor a 10, el sistema debe mostrar un mensaje de error y pedir nuevamente el dato.

5. No se puede crear un ticket si ya existe un ticket activo.

6. El sistema debe verificar si la variable ticket_activo es verdadera antes de permitir crear un nuevo ticket.

7. No se puede crear un ticket si el parqueo está lleno.
8. El sistema debe comparar los espacios ocupados con la capacidad total del parqueo.
9. La placa del vehículo debe tener entre 6 y 8 caracteres sin espacios.
10. Si la longitud de la placa está fuera de ese rango, el sistema debe solicitar nuevamente el dato.
11. El tipo de vehículo debe ser válido.

12. El sistema solo debe aceptar las siguientes opciones:
    1 = Moto
    2 = Auto
    3 = Pickup o SUV
13. Solo se puede registrar salida si existe un ticket activo.

14. Si no hay ticket activo, el sistema debe mostrar un mensaje indicando que no hay vehículos en el parqueo.

15. Los minutos para simular el paso del tiempo deben estar entre 1 y 1440.

16. Si el número está fuera de ese rango, el sistema debe mostrar un error y pedir nuevamente el valor.

17. Si el tiempo estacionado es menor o igual a 15 minutos, el cobro es gratuito.

18. Si el tiempo estacionado supera las 6 horas, se aplica una multa fija de Q25.

19. Si el cliente es VIP, se aplica un descuento del 10% sobre el monto calculado.

20. Si el tiempo estacionado supera las 12 horas, se aplica un recargo adicional del 20% sobre el monto final.

Cálculos:

1. Calculo de espacios ocupados:
   Espacios ocupados= Tickets creados-tickstes cerrados

2. Cálculo de espacios disponibles:
   Espacios disponibles= Capacidad- Espacios ocupados

3. Cálculo del tiempo estacionado;
   Minutos estacionado= Tiempo actual-Hora de entrada

4. Conversión de minutos a horas:
   Horas= Minutos estacionado/60

5. Cálculo del monto base:
   Monto base= Horas*tarifa según vehículo:
   a. Moto: Q5 por hora
   b. Carro: Q10 por hora
   c. SUV/Pickup: Q15 por hora

6. Multa por mas de 6 horas:
   monto=monto+25

7. Descuento VIP:
   Descuento= monto*0.10
   Monto final= monto-descuento

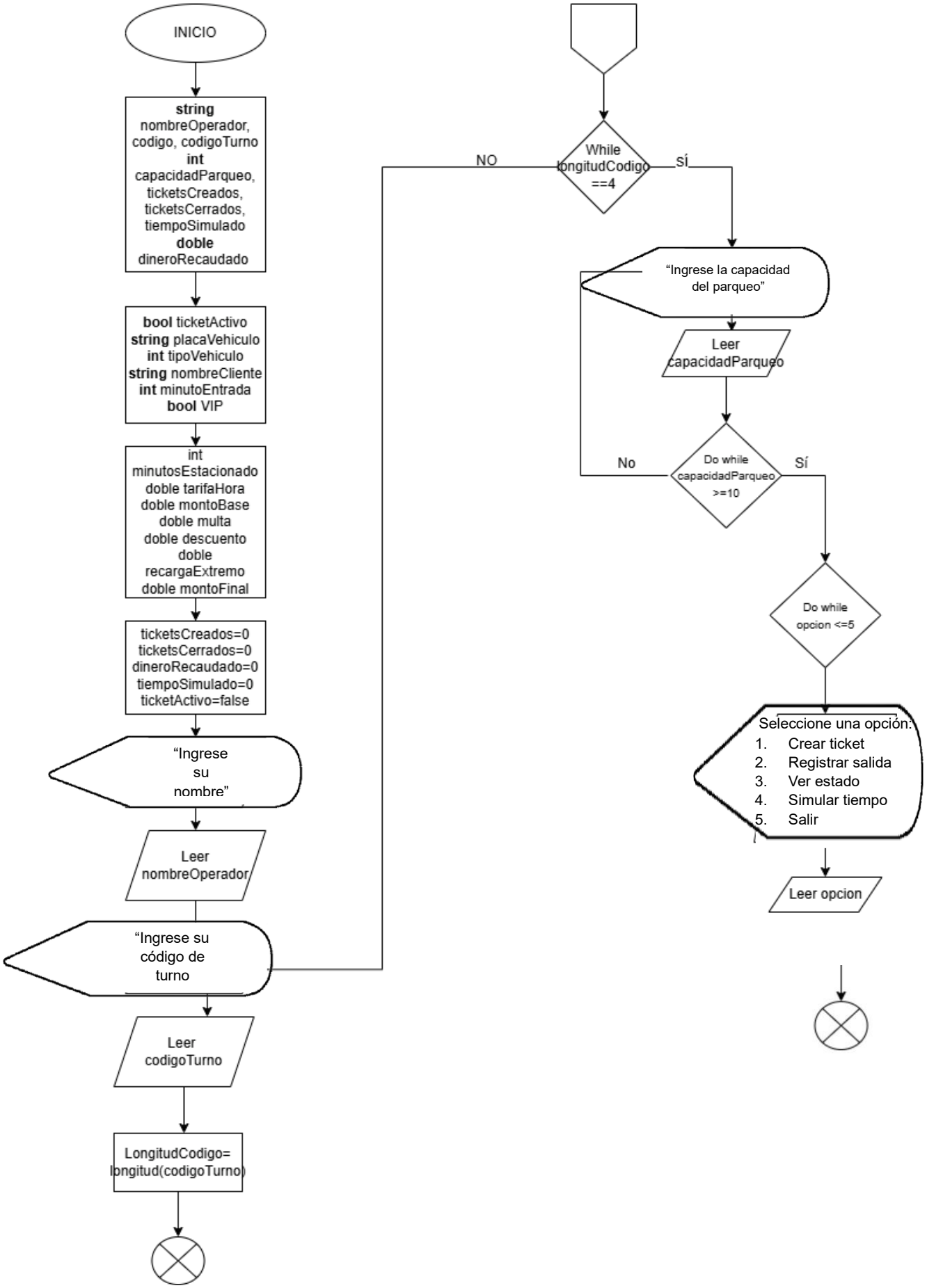
8. Recargo por permanencia extrema:
   recargo=Monto*0.20
monto final= monto+recargo

9. Dinero recaudado: dinero recaudado=dinero recaudado + monto final

10. Actualización de tickets cerrados
    Tickets cerrados= tickets cerrados+1

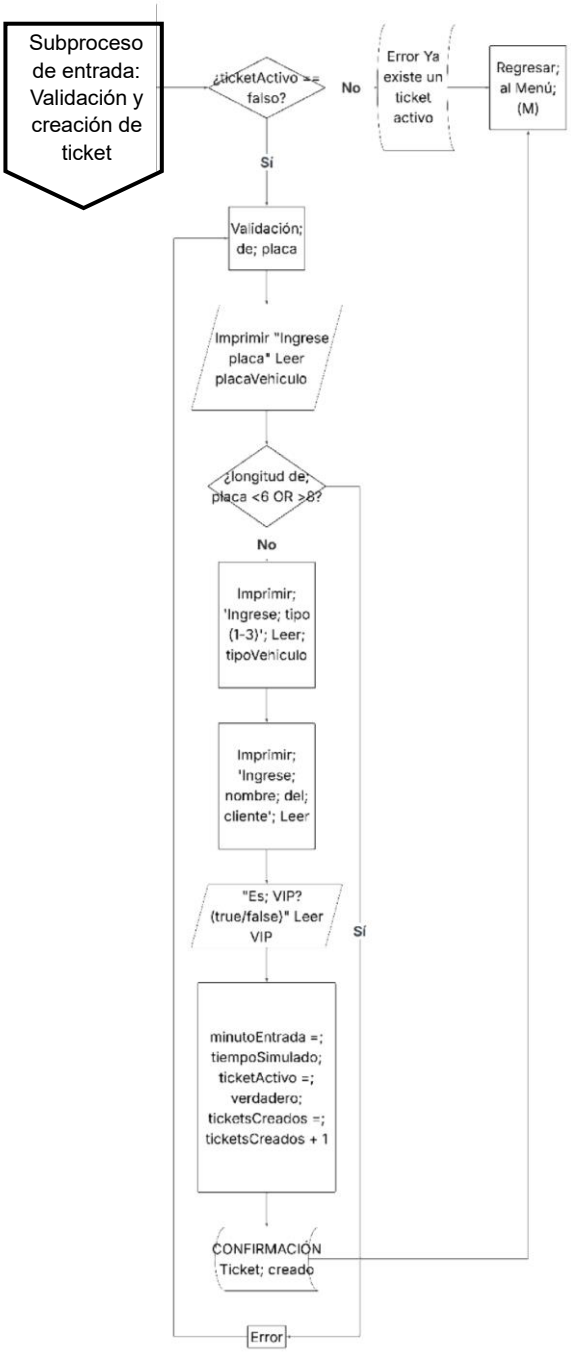
5. ALGORITMO

Diagramas de flujo


INICIO
string nombreOperador, codigo, codigoTurno int capacidadParqueo, ticketsCreados, ticketsCerrados, tiempoSimulado doble dineroRecaudado
bool ticketActivo string placaVehiculo int tipoVehiculo string nombreCliente int minutoEntrada bool VIP
int minutosEstacionado doble tarifaHora doble montoBase doble multa doble descuento doble recargaExtremo doble montoFinal
ticketsCreados=0 ticketsCerrados=0 dineroRecaudado=0 tiempoSimulado=0 ticketActivo=false
"Ingrese su nombre"
Leer nombreOperador
"Ingrese su código de turno
Leer codigoTurno
LongitudCodigo= longitud(codigoTurno)
While longitudCodigo ==4
NO
sí
"Ingrese la capacidad del parqueo"
Leer capacidadParqueo
Do while capacidadParqueo >=10
No
Sí
Do while opcion <=5
Seleccione una opción:
1. Crear ticket
2. Registrar salida
3. Ver estado
4. Simular tiempo
5. Salir
Leer opcion

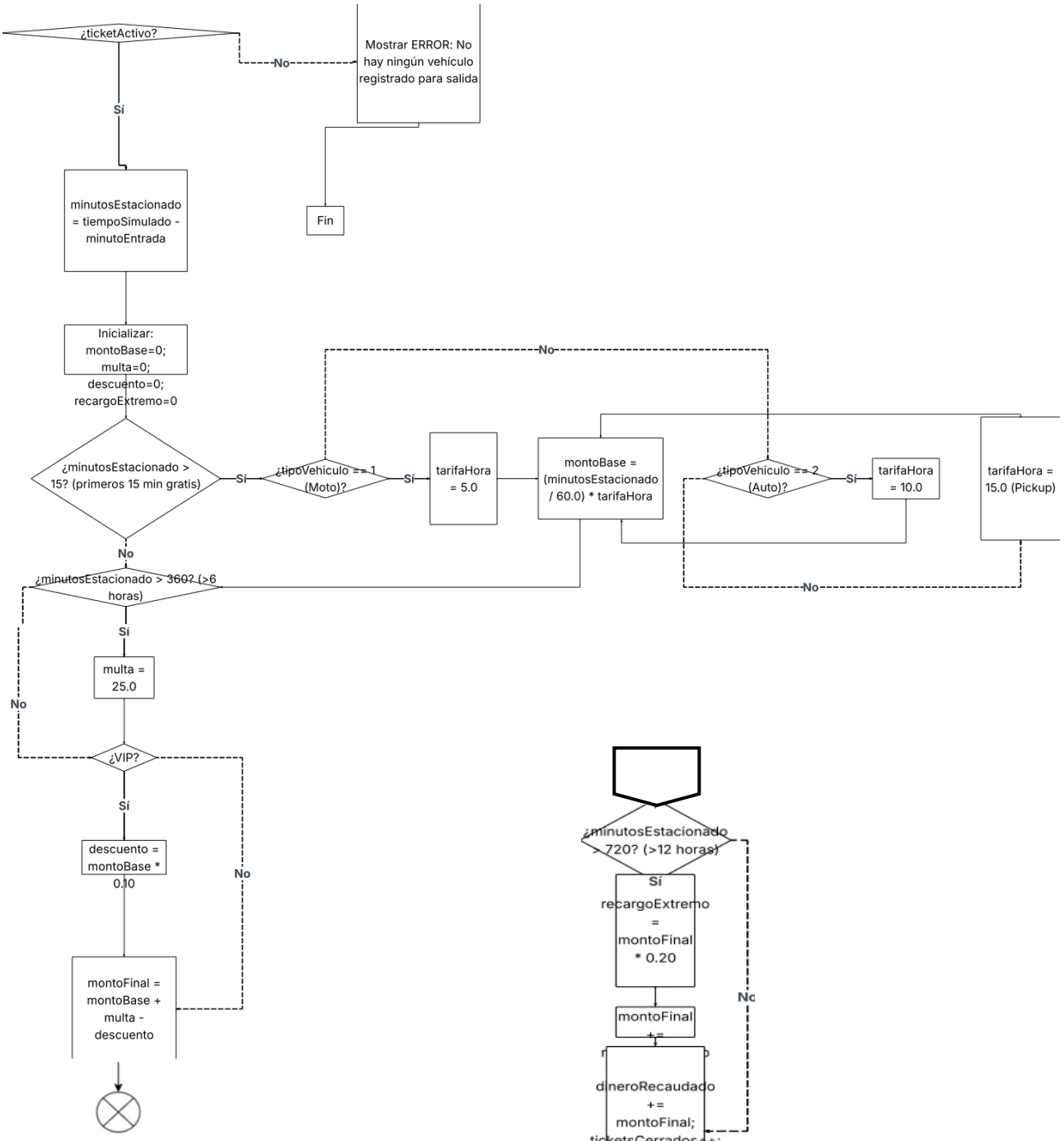
CASE 1:

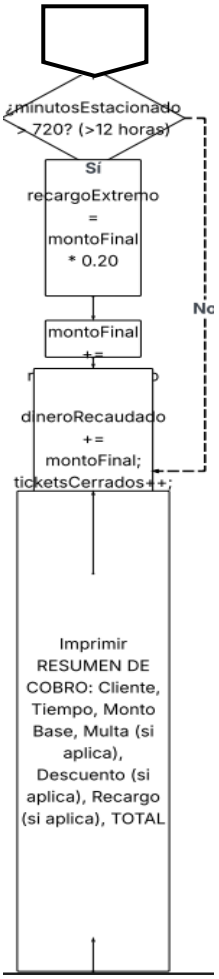
CREAR TIC



CASE 2:

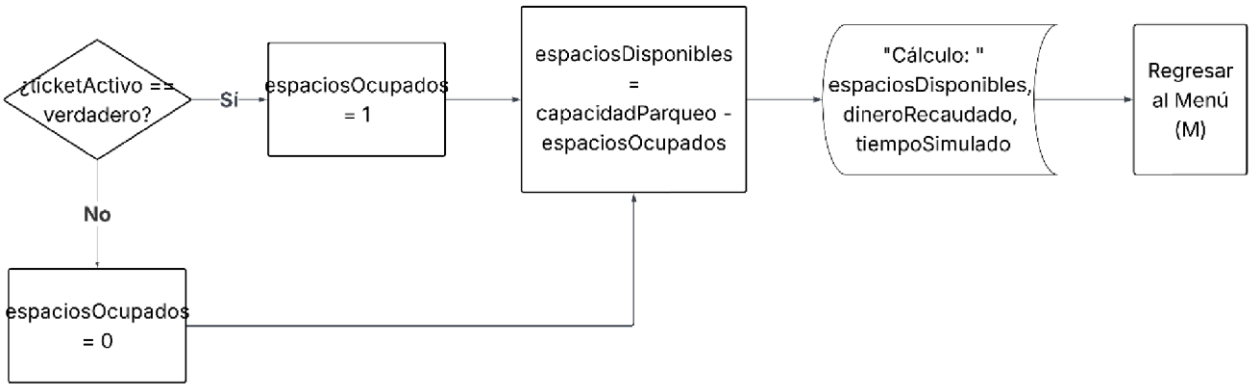
SALIDA Y COBRO





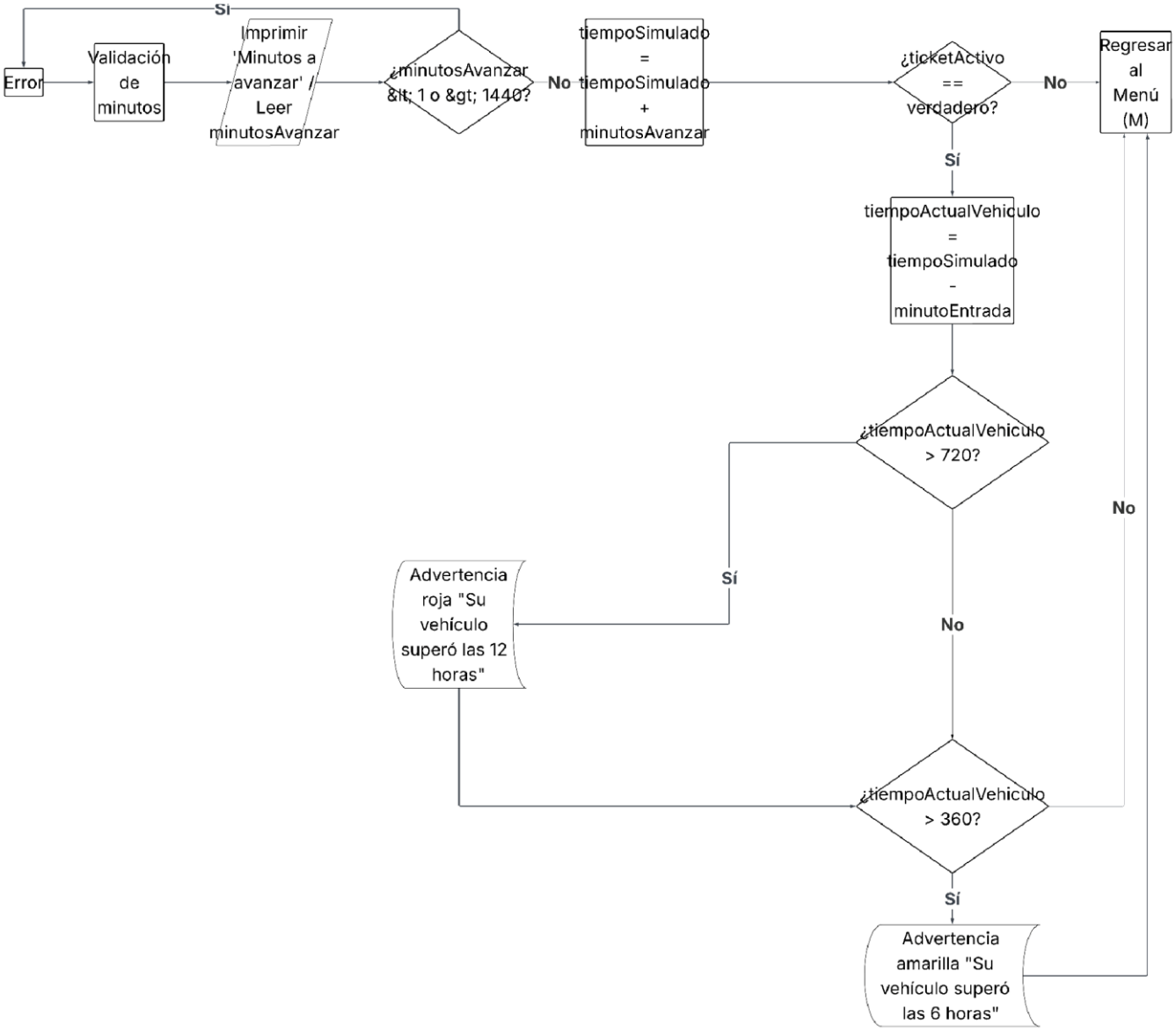
CASE 3:

VER ESTADO DE PARQUEO

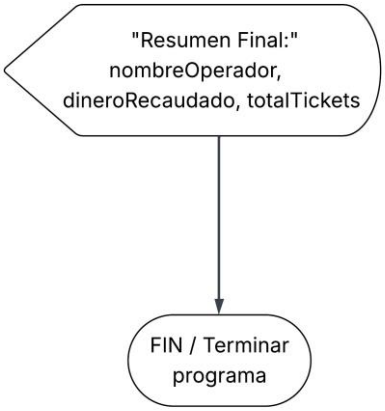


CASE 4:

CASE 4: SIMULAR TIEMPO



CASE 5: SALIR Y RESUMEN



ALGORITMO

Pseudocódigo

```
MAIN {     string nombreOperador, codigoTurno, placaVehiculo, nombreCliente     int
capacidadParqueo, ticketsCreados = 0, ticketsCerrados = 0, tiempoSimulado = 0
int tipoVehiculo, minutoEntrada = 0, minutosEstacionado, opcion, longitudCodigo     int
longitudPlaca, minutosAvanzar, espaciosOcupados, espaciosDisponibles,
tiempoActualVehiculo

   double dineroRecaudado = 0.0, tarifaHora = 0.0, montoBase, multa, descuento,
recargoExtremo, montoFinal     bool ticketActivo = false, VIP

   PRINT "Ingrese nombre del operador:"
   READ nombreOperador;
    do
{
       PRINT "Ingrese codigo de turno (4 caracteres):"
READ codigoTurno
       longitudCodigo = LENGTH(codigoTurno)        if
(longitudCodigo != 4) PRINT "ERROR: Codigo invalido"
   } while (longitudCodigo != 4)
    do
{
       PRINT "Ingrese capacidad (minimo 10):"        READ capacidadParqueo
if (capacidadParqueo < 10) PRINT "ERROR: La capacidad minima es 10."     }
while (capacidadParqueo < 10)
    do
{
       PRINT "------ MENU PRINCIPAL (Reloj: " + tiempoSimulado + " min) ------"
       PRINT "1. Crear ticket de entrada"
       PRINT "2. Registrar salida y calcular cobro"
       PRINT "3. Ver estado del parqueo"
       PRINT "4. Simular paso del tiempo"
       PRINT "5. Salir"
       READ opcion

       switch (opcion) {            case
1:                if (ticketActivo ==
false) {                    do {
                      PRINT "Ingrese placa (6 a 8 caracteres):"                    READ
placaVehiculo                    longitudPlaca = LENGTH(placaVehiculo)                    if
(longitudPlaca < 6 || longitudPlaca > 8) PRINT "ERROR: Placa invalida."
                  } while (longitudPlaca < 6 || longitudPlaca > 8)

                  PRINT "Tipo (1=Moto, 2=Auto, 3=Pickup):"
                  READ tipoVehiculo
                  PRINT "Nombre del cliente:"
                  READ nombreCliente
```

```
                PRINT "Cliente VIP? (true/false):"
                READ VIP

                minutoEntrada = tiempoSimulado
ticketActivo = true
                ticketsCreados = ticketsCreados + 1
                PRINT "CONFIRMACION: Ticket creado."
            } else {
                PRINT "ERROR: Ya existe un ticket activo."
            }
break
          case
2:
            if (ticketActivo == true) {
                minutosEstacionado = tiempoSimulado - minutoEntrada
montoBase = 0; multa = 0        descuento = 0
                recargoExtremo = 0

                if (minutosEstacionado <= 15) {
montoBase = 0
                } else {                        if (tipoVehiculo == 1)
tarifaHora = 5                        else if (tipoVehiculo == 2)
tarifaHora = 10                         else if (tipoVehiculo == 3)
tarifaHora = 15

                    montoBase = (minutosEstacionado / 60.0) * tarifaHora
                }

                if (minutosEstacionado > 360) multa = 25
if (VIP == true) descuento = montoBase * 0.10

                montoFinal = montoBase + multa - descuento

                if (minutosEstacionado > 720) {
recargoExtremo = montoFinal * 0.20
montoFinal = montoFinal + recargoExtremo
                    PRINT "ADVERTENCIA: Recargo por permanencia prolongada."
                }

                dineroRecaudado = dineroRecaudado + montoFinal
ticketsCerrados = ticketsCerrados + 1                    ticketActivo =
false
                PRINT "Cobro final: Q" + montoFinal
            } else {
                PRINT "ERROR: No hay ticket activo."
            }
break
```

```
            case
3:
                if (ticketActivo == true) espaciosOcupados = 1; else espaciosOcupados = 0
espaciosDisponibles = capacidadParqueo - espaciosOcupados

                PRINT "Capacidad total: " + capacidadParqueo
                PRINT "Ocupados: " + espaciosOcupados + " | Disponibles: " +
espaciosDisponibles
                PRINT "Tiempo simulado: " + tiempoSimulado + " min"
                PRINT "Total recaudado: Q" + dineroRecaudado
                PRINT "Tickets creados: " + ticketsCreados + " | Cerrados: " + ticketsCerrados
break
            case
4:
do {
                    PRINT "Minutos a avanzar (1 a 1440):"                    READ
minutosAvanzar                    if (minutosAvanzar < 1 || minutosAvanzar > 1440)
PRINT "ERROR: Valor invalido."
                } while (minutosAvanzar < 1 || minutosAvanzar > 1440)

                tiempoSimulado = tiempoSimulado + minutosAvanzar
                PRINT "Tiempo acumulado: " + tiempoSimulado + " min"
                 if (ticketActivo == true) {                    tiempoActualVehiculo
= tiempoSimulado - minutoEntrada                    if
(tiempoActualVehiculo > 720) {
                        PRINT "ADVERTENCIA: Recargo extremo proximo."
                    } else if (tiempoActualVehiculo > 360) {
                        PRINT "ADVERTENCIA: Multa proxima."
                    }
}
break
            case
5:
                PRINT "--- RESUMEN FINAL ---"
                PRINT "Operador: " + nombreOperador
                PRINT "Recaudado: Q" + dineroRecaudado
                PRINT "Tickets Creados: " + ticketsCreados + " | Cerrados: " + ticketsCerrados
PRINT "Tiempo Simulado Total: " + tiempoSimulado + " min"                break


            default:
                PRINT "ERROR: Opcion no valida."
                break;
        }

    } while (opcion != 5)
}
```